IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Probing Video Diffusion U-Nets for Content vs. Motion

Team Name: Claudn't *Team Members: [roll numbers]*

Abstract

A text-to-video diffusion U-Net alternates spatial layers, which see one frame, with temporal layers, which see all frames. We asked where inside such a model the content of a clip (what is depicted) and its motion (how it moves) are actually decided, and whether that answer is good enough to edit one without the other. We used AnimateDiff (Stable Diffusion 1.5 with a motion adapter) and generated 360 clips in 750 controlled pairs in which a single prompt word changes and the initial noise is fixed. From these pairs we built selectivity maps over 16 layers and all 25 denoising steps, and tested three hypotheses whose test statistics were fixed before the data existed. Motion selectivity is weak almost everywhere; it peaks in the temporal module of the first decoder block (`up_blocks.0`, 8×8 latent) in 1000 of 1000 bootstrap resamples, and only during the first 2–3 of 25 steps (changepoint $\tau = 2$, 95% CI [1,3]). We then turned the static/dynamic split used for the maps into an intervention, a *component patch* that transplants only the frame-varying part $\tilde{x}$ or only the frame mean $\bar{x}$ of an activation between two generations. On 12 held-out test pairs the selected component patches beat a prompt-only edit on the median in both directions (median ΔD +0.041 and +0.068) and beat whole-tensor patching, but their 95% confidence intervals include zero, so our pre-registered success criterion is not met. Temporal-attention injection, the training-free baseline, is the only method whose interval excludes zero: +0.105 (CI [0.001, 0.204]) over the first third of the steps, and +0.120 (CI [0.020, 0.203]) over all steps. We also trained motion LoRAs placed by the map and in a deliberately wrong place. Across three seeds the map-guided LoRA is the only one with a positive median gain in every seed, though the gain is small (+0.009 to +0.015) and not significant. We also built a Gradio interface for live edits, uploaded videos and browsing every scored clip.

1 Introduction

Recent text-to-video diffusion models can easily turn a prompt into a short, realistic clip. Every such clip combines two things i.e. its *content* which denotes what is present in the frames, like a dog etc. , and its *motion* which denotes how the content present in the frames is moving. In principle the two are independent in the sense that the same motion could be performed by a different object. For example instead of a dog running along a road you could have a horse running along the road. This project asks that at whether a video diffusion model keeps them apart internally and if it does then in which layers and at which denoising steps does it decide content and and ata which steps does it decide the motion, and whether that knowledge is precise enough to change one while keeping the other.

Answering this question is important because people increasingly want to edit a generated clip rather than regenrating it because changing the prompt changes the subject, the background and motion all at once. Recent video diffusion models are built on image diffusion models by adding temporal layers to them, as AnimateDiff [4] does with Stable Diffusion [13]. Recent editing methods [17; 21; 10; 18] intervene on chosen layers or features to transfer or customise motion, but they chose those places by design or by ablation. The obvious reading by archtecture stating spatial and temporal layers is usually assumed rather than measured. A measured mapping of where the content lies and where the motion lies would allow targetted edits, changing only certain parts of the clip and retaining the other desired ones rather than entire regeneration of the clip. It would show us where to place the few trainable parameters of LoRA [7] when

teaching the model a new motion. A negative answer is also useful because if the split is not clean then the editing methods that assume this clean split are working for some other reasons that need further invetsigation.

To find such a map, we experiment with the model. The model is a U-net architecture that turns noise into video over a sequence of denoising steps. It was trained on just the input and the output and was never told as to which part of the model should carry the motion and which should carry the content. So any such seperation should be something that it learned on its own and has to be located emperically because it is spread across many high-dimensional layers and across many denoising steps so it cannot be read by weights alone nor by the architecture diagram. Architectural diagrams labelling layers as spatial and temporal tells us what each layer *can* see not what it actually ends up encoding. Finding the seperation would require us to actually study the trained network by changing the input in a controlled way and then observing how the activations are responding to those changes and then testing whether the places we found really control the desired properties or by intervening on them midst generation. We then use evalutations to judge whether the edits were successful or not.

The report is organised as follows. Section 2 lists how the work progressed. Section 3 surveys the work we built on. Section 4 describes the methods, split into work done before and after the prep-presentation review. Section 5 covers the data, Section 6 the experiments and their settings, and Section 7 the results. Section 8 gives the plan for novelty assessment and Section 9 concludes.

2 Project Workflow

We started with the question of finding out where the content lives and where the motion lives. The architecture suggests an easy answer that the appearance would be stored in the spatial layers and the motion would be stored in the temporal layers. But we wanted to test it out by measuring it. Our first experiments showed that the motion was harder to measure than we expected because pixels that move also change depending on the object performing the motion. We developed tools to look inside the model when it generated clips and used those measurements to gauge where is content stored and where is motion stored. And based on those observations we tried to edit the model mid generation to validate our hypthesised places of residence of content/motion. Some of our predictions were correct and some of them turned out to be false that told us what to try out next.

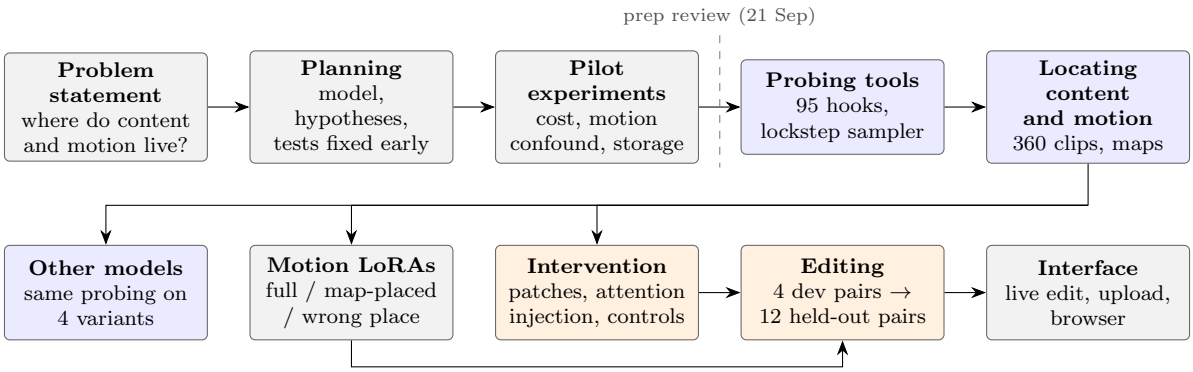


Figure 1: Order of the work. Everything left of the dashed line was done before the prep-presentation review. The maps from the probing decide where we intervene and where the map-placed LoRA sits, and the same probing is then repeated on four other model variants.

- **Problem framing:** Problem was to find out which layers store content and which of them store

motion of a video diffusion U-net model. Then use this information to edit a clip so that one of these is kept intact and the other is changed.

- **Planning:** We defined what content meant and what motion meant so we could measure it properly, chose AnimateDiff as our model over ModelScope and Stable Video Diffusion because it has explicit spatial and temporal split and takes text prompts as input and we fixed our prompt, the variables we would change and three hypothesis that we would test before seeing any data.
- **Experiments:**
  - Measured the cost of generating each clip
  - Found that the pixel space motion also depends on the object we are using to perform the motion which gave us the idea to measure motion on the residual of the frame mean for each frame
  - Found out that storing the full activations required way too much memory so we scored the pairs during generation itself.
- **Probing Tools:** Built a package that hooks into 95 modules of the U-net, wrote a sampler that generated groups of clips thata were bit identical to the diffusers pipeline and built a runner for the Kaggle T4s.
- **Locating content and motion:** Generated 360 clips that formed 750 pairs with the one word swaped in the prompt. Then selectivity mas were created for each layer and steps. We tested Our 3 hypothesis and trained the linear probes.
- **Intervention Framework:** Built patches that transplant only the static or only the frame varying part of the activation from one generation to another. Also built a temporal attention injection baseline.
- **Editing:** Chose one configuration per direction on 4 pairs of development and scored it on 12 pairs. Added attention level edits and prompt switching.
- **Trained Model:** Trained motion LoRAs on where the map points and also where it does not. After finding scene cuts in training clips we retrained.
- **Check findings on other models:** We repeated the probing on 4 other model variants to see if the map is specific to the diffusion model we used or does it apply in general.
- **Interface:** A live app where we can edit an uploaded video and view the maps and cross-attention

3 Literature Survey

Video is a stack of static frames that contain content. A video model hence would require two kinds of layers, one that is spatial in nature that stores the content and one that is temporal in nature and stores the motion.

Video Diffusion Models [6] built a video model this way. They extended the image U-Net to video generation by factorising attention into a spatial part and a temporal part. Video LDM [2] did the same in the latent space of Stable Diffusion [13] by keeping its weights fixed and then inserted the temporal layers and trained only the new layers. AnimateDiff [4] further built on this idea. People have their own fine tuned image models each of which generated images in a particular style. AnimateDiff's goal was to make all of these image models produce videos without needing them to train each time the image model is changed. He aimed to train the motion module once and then attach it to any image model to generate videos.

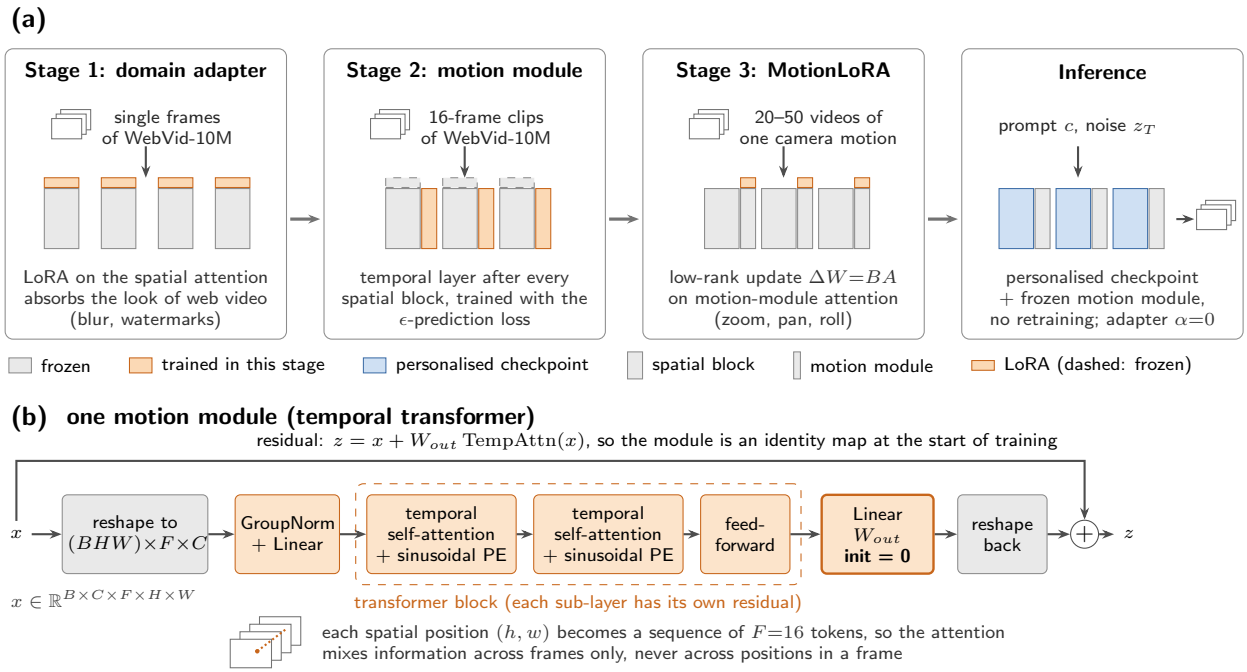


Figure 2: AnimateDiff [4]. (a) The three training stages and inference: a domain adapter (LoRA on the frozen image U-Net), the motion modules (the only weights trained on video), and MotionLoRA; at inference the motion module is attached to a personalised checkpoint. (b) One motion module: the activation is reshaped so that the F frames at each spatial position form a sequence, passed through temporal self-attention, and added back to its input through a zero-initialised output layer.

Motion module This refers to a small attention block after the spatial layer. It takes 16 frames at one pixel position and applies attention to it with other frames at the same position. A position encoder here helps the model know the order of frames. The output gets added back to its input and its last layer starts with all zero weights so that when the model is being trained it behaves excatly like an image model. During training only the motion modules are trained not changing the image models weights. **Domain adapter** Web videos are blurry and they might carry watermarks. Before training the motion module. they trained a small add-on to the image layers that soaks up this look so the motion module would not learn those things. That portion is switched off during inference. **MotionLoRA** A small trainable add-on LoRA was added to the motion modules attention. It teaches a new camera movement from a small number of videos. The motion module adds its output on top of whatever comes in so the activations after it should hold content and motion mixed together. Copying the motion module's output from one generation to another copies both of them, which is why we just copy a part of it. The motion modile already treats the 16 frames as a sequence so it becomes very natural to split the activations into a part that is constant over every frame and a part that varies with every frame. We chose this model because the weights that encode motion and content are physically seperate and moreover they are even trained seperatrely. If even this model mixes motion and content then the models that are trained collectively would certainly mix them up.

Now knowing which layers are temporal tells us where the motion could be stored not what those layers actually have there. To find that out we needed a way to pull out the motion activation which MOFT gave us.

Xiao et al. [18] found that features of video diffusion U-net mix content and motion with most of the chan-

nels carrying content. They extract motion in two steps: first they subtract the mean over the frames $X - \frac{1}{F}\sum_{i=1}^{F} X_i$, which removes the part the remains constant over the frames, and then keep the top 4% channels chosen by PCA. The result is called MOFT and it encodes the direction of motion. They read it after the decoder block and use it to guide the generation of a new video without training by guiding the latents untill their MOFt matches a refrence. Our split is thier first step and our hypothesis comes from this paper. Our component patch copies the frame-varying part of the activation from one generation to another. MOFT shows where the motion can be read from the features but not whether the model actually uses it there. To test that we use activation patching.

Activation patching. Meng et al. [11] located factual associations in GPT by running the model on a corrupted input and restoring one hidden state at a time from a clean run (causal tracing); a state whose restoration recovers the answer is causally implicated. Wang et al. [16] used path patching to recover a full circuit for a single task in GPT-2 small. Zhang and Nanda [20] showed that the conclusions depend on choices that are easy to make carelessly: how the input is corrupted, which metric is used (probabilities versus logit differences), and whether one patches a single layer or a window. We took three things from this line of work. We patch clean activations from one generation into another rather than adding noise. We use windows of denoising steps as well as single steps. And we run null and mis-indexed patches as controls before trusting any effect.

Motion inside video diffusion features. Xiao et al. [18] found that removing the frame-mean component of a video diffusion U-Net's features, followed by a channel selection, leaves a representation (MOFT) that encodes motion direction and can guide generation without training. Their best features came from just after the first up-block. This is the origin of our static/dynamic split and of hypothesis H1. MotionClone [10] transfers motion from a reference video by guiding the temporal attention maps of the generated video towards the reference's; Diffusion Motion Transfer [19] does the same with a loss on space-time features. Both show that temporal attention carries enough motion to transfer it. Our attention-injection baseline is MotionClone's core idea in its simplest form: we replace the attention probabilities instead of guiding towards them with gradients, which the brief's compute budget could not afford.

Motion customisation by fine-tuning. Tune-A-Video [17] and MotionDirector [21] adapt a video model to a reference motion. MotionDirector uses a dual-path LoRA [7], one on the spatial layers for appearance and one on the temporal layers for motion, so that the motion can be reused with new content. Our trained model borrows the motion half and asks a narrower question: does a LoRA placed only where our map says motion lives do as well as one placed everywhere?

Attention as an explanation in image diffusion. Prompt-to-Prompt [5] edits images by reusing the cross-attention maps of one generation in another, and DAAM [15] turns cross-attention into word-level heat maps. Both treat attention maps as the place where layout is decided. Our timestep hypothesis H2 comes from the common claim that diffusion models settle layout and motion early and appearance late, which Baherwani et al. [1] tested for video diffusion, finding two regimes over the denoising trajectory with a fairly consistent boundary between them.

4 Proposed Approach

4.1 Model and notation

AnimateDiff consists of the frozen Stable Diffusion 1.5 U-Net (about 860 M parameters) with the motion adapter `guoyww/animatediff-motion-adapter-v1-5-2` [22] (about 417 M parameters) inserted after the spatial layers of every block. The U-Net works on 4-channel latents of 64×64 (for 512×512 frames) and has four encoder blocks, a middle block and four decoder blocks (Figure 3). Each spatial transformer ap-

plies self-attention within a frame and cross-attention to the CLIP text embedding. Each motion module reshapes the activation so that the 16 frames become the sequence dimension, applies two temporal self-attention layers with sinusoidal position encoding, and adds the result back to its input. There are 21 motion modules (42 temporal attention layers) and 16 spatial transformer blocks. We call one hookable output at one denoising step a *coordinate*; the model has 95 hookable modules.

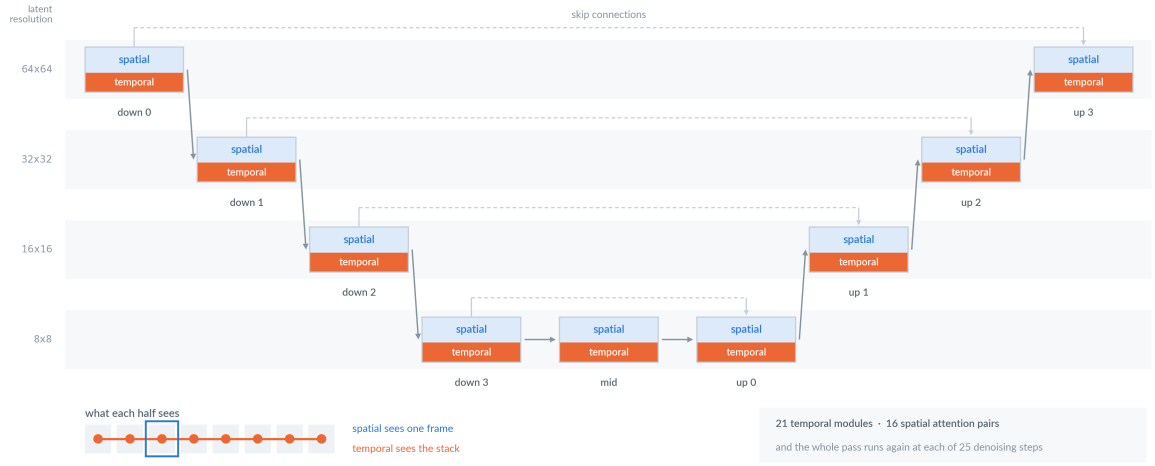


Figure 3: The AnimateDiff U-Net. Every block pairs spatial layers (one frame at a time) with a temporal motion module (all 16 frames at one position). The whole pass runs at each of the 25 denoising steps. Block names in the text follow diffusers (`down_blocks.0` … `up_blocks.3`); `up_blocks.0` is the 8×8 decoder block right after the bottleneck.

Generation is fixed throughout (configuration frozen before the sweep): 16 frames, 512×512, 25 DDIM steps [14], classifier-free guidance scale 7.5, fp16, and the noise tensor drawn on the CPU from a seed so that it is identical across machines.

For an activation at one coordinate we write it in a canonical form $x \in \mathbb{R}^{F \times C \times S}$ (frames, channels, spatial positions) and split it into

$$\bar{x} = \frac{1}{F} \sum_{f=1}^{F} x_f, \qquad \tilde{x} = x - \bar{x}, \tag{1}$$

the static part, identical in every frame, and the frame-varying residual. For two clips a, b generated from the same noise, with one prompt word different, the distance at a coordinate is $d = 1 - \cos(a, b)$ on the flattened tensor, computed separately on $\bar{x}$ and on $\tilde{x}$.

4.2 Work done before the prep-presentation review

Choice of model. We compared three open models. AnimateDiff has an explicit spatial/temporal split in its module tree, mature diffusers support, and fit on a T4 (10.2 GB measured). ModelScope T2V (1.7 B parameters) trains space and time together end to end, which makes it the harder and more interesting case, but the tooling for extracting its attention is thinner. Stable Video Diffusion has the best output quality but is conditioned on an image only, so we could not build matched pairs from one-word prompt changes. We chose AnimateDiff for all deliverables and kept ModelScope as a later robustness check.

Definitions and protocol. From the brief we fixed: content is what an image classifier would see in a single frame (subject identity, colour, texture, layout); motion is how pixels move across frames, regardless

of what they belong to. We wrote a roadmap with a register of 22 decisions (D-01 to D-22), among them the distance (D-01: $1 - \cos$), the static/dynamic split (D-02), how pairs are formed (D-05), the edit metric and success criterion (D-14, D-16), and the baseline (D-17).

Three hypotheses with fixed tests. Each one predicts something that could fail.

- **H1 (layer).** Motion selectivity peaks in a medium-resolution decoder block (`up_blocks.1`), following [18]. Test: the bootstrap argmax over the nine temporal blocks of $M(b) = -\text{mean}_t \,\text{median}\, S_{dyn}(b, t)$ must exclude the extreme-resolution blocks.
- **H2 (timestep).** Motion is settled early in denoising. Test: a single least-squares changepoint τ on the winning block's profile must fall in the first half, with the early segment above the late one.
- **H3 (representation).** The frame mean $\bar{x}$ is content-dominated, and removing it reveals motion selectivity that the raw activation hides. Test: at least 75% of coordinate-steps have S_{stat} significantly content-selective, and at least one temporal coordinate flips from content-selective in $\bar{x}$ to motion-selective in $\tilde{x}$.

Pilots E1–E3 (Kaggle T4).

- **E1, cost.** 94.1 s and 10.2 GB per clip on a T4 at our settings, so the full 360-clip probing set needs about 9.4 GPU-hours.
- **E2, does a fixed noise tensor isolate a word?** Seven clips from one noise tensor: "a golden retriever running on a beach", three subject swaps and three verb swaps. Subject swaps moved CLIP appearance 2.6× more than verb swaps, so the content side of the control works. The motion side failed. A frame-difference motion proxy moved *more* under subject swaps (0.461) than under verb swaps (0.390), because frame differences sit on the silhouette of the object and a new object has a new silhouette (Figure 4). Pixel motion is confounded by content. This is why selectivity is measured on the frame-mean-removed residual, and why content pairs only swap subjects within a body-plan family (a dog for a wolf, not a dog for a person).
- **E3, can we get the activations out?** Hooks on all 95 modules cost 0.6% of generation time if they only record metadata. A full fp16 dump of every coordinate at every step is 87.9 GiB per clip; a summary (channel statistics, per-frame residual norms, one 16×16 map per attention layer) is 44.6 MiB, 2,017× smaller. Storing full activations for 360 clips was therefore impossible, which forced the online pair scoring described next.

4.3 Work done after the prep-presentation review

Lockstep sampler and online scoring. A selectivity map needs, for every pair, the distance between two clips' activations at every coordinate and step. Caching the anchor clip's activations at all 25 steps for 16 coordinates takes about 7.6 GB of host memory per anchor, and a group has 21 anchors. We instead denoise all 60 clips of a (seed, scene) group *in lockstep*: at step t every anchor runs and its step-t activations are kept, then every partner runs and is scored against them, then the step-t activations are dropped. Memory holds one step instead of 25, every clip is generated exactly once, and the distances are computed on the full tensors rather than on summaries. The loop re-implements diffusers' `AnimateDiffPipeline` step by step (CFG concatenation, prompt-embedding repetition over frames, the DDIM update). We verified it on the GPU before use: a clip generated alone, inside a group, and with all probes attached gives a final latent identical to `pipe()` (mean absolute difference exactly 0).

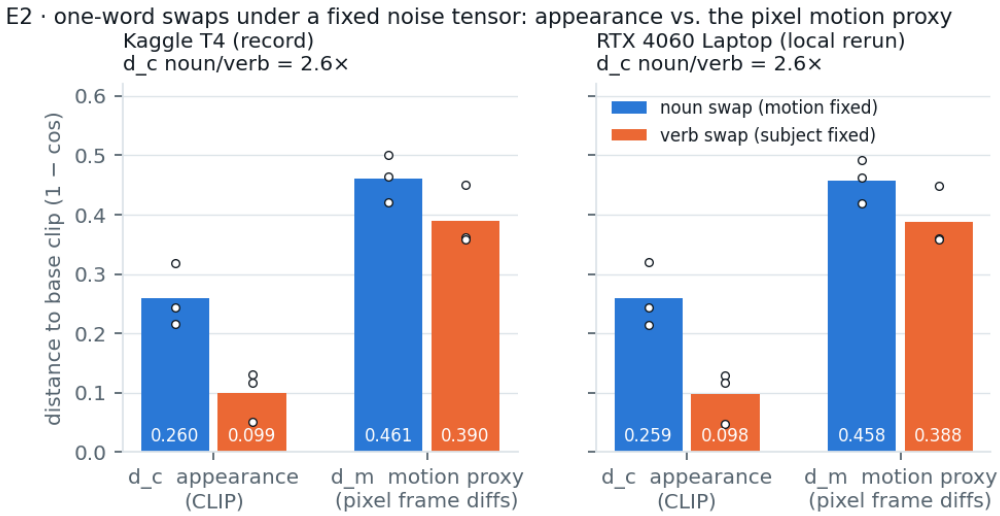


Figure 4: Pilot E2. A pixel-space motion proxy changes more when the subject is swapped (motion held fixed) than when the verb is swapped. Measuring motion in pixel space would have mistaken content for motion.

Selectivity. For one coordinate (L, t), let d_c be the distance across a content pair (subject swapped) and d_m across a motion pair (verb swapped). Then

$$S = \frac{d_c - d_m}{d_c + d_m} \in [-1, 1], \tag{2}$$

positive when the coordinate tracks what is depicted and negative when it tracks how things move. S_{dyn} uses $\tilde{x}$ and S_{stat} uses $\bar{x}$. Since d_c and d_m come from different pairs, we compute S on matched units: a clip B = (subject s, verb v, scene) that is the end point of both a content pair (family anchor, v) $\to B$ and a motion pair (s, running) $\to B$. This gives 78 units per seed. Each map cell is the median over units with a 1,000-sample bootstrap CI within each seed, averaged over the three seeds.

Linear probes. To check whether motion information is *present* even where selectivity is low, we trained logistic-regression probes for the verb (4 classes) and the subject's family (2 classes) on each coordinate and step. Storing full tensors was impossible, so each clip's $\tilde{x}$ and $\bar{x}$ were compressed with a count sketch to 512 dimensions, a random projection that preserves inner products in expectation. Pipeline: standardise, PCA fitted on the training split only, logistic regression ($C = 1$). We trained on one scene and tested on the other, so a probe cannot succeed by memorising the background.

Component patching. A patch replaces part of the target generation's activation at a coordinate with the source generation's. Besides the usual whole-tensor, channel-subset and α-scaled patches, we added a *component* patch that moves only one term of Eq. (1):

$$\text{dynamic:}\quad x_{out} \leftarrow \bar{x}_{tgt} + \tilde{x}_{src}, \qquad \text{static:}\quad x_{out} \leftarrow \bar{x}_{src} + \tilde{x}_{tgt}. \tag{3}$$

The motion modules write into the residual stream, so a whole-tensor patch there copies content and motion together. The split that defines the map is the natural way to move one and keep the other. In fp16 the obvious implementation $\bar{x}_{out} + x_{src} - \bar{x}_{src}$ is not an exact identity when source and target are the same clip; rounding errors grew over 25 steps to a mean absolute difference of 10^{-2}. We compute it instead as $x_{out} + (\delta - \bar{\delta})$ with $\delta = x_{src} - x_{out}$, which is exactly zero for a self-patch.

Temporal attention injection (baseline). In each temporal attention layer the target uses the source's attention probabilities P_{src} instead of its own over a chosen window of steps, with its own values V. To

Algorithm 1 One edit: source A and N target arms denoised in lockstep from one noise tensor

Require: noise z_T, prompts p_A (source) and p_{tgt}, arms $\{(\text{coords}_k, \text{steps}_k, \text{component}_k, \alpha_k)\}_{k=1}^{N}$

$z^A \leftarrow z_T$; $z^k \leftarrow z_T$ for $k = 1..N$
for t in the 25 DDIM steps **do**
  run U-Net on z^A with p_A; cache A's outputs at every coordinate any arm patches at t
  for $k = 1..N$ **do**
    run U-Net on z^k with p_{tgt}; at each $c \in \text{coords}_k$, if $t \in \text{steps}_k$:
      $\delta \leftarrow x_c^A - x_c^k$; $x_c^k \leftarrow x_c^k + \alpha_k \cdot \text{part}(\delta)$ ▷ part $= \delta - \bar{\delta}$ (dynamic), $\bar{\delta}$ (static) or δ (whole)
    $z^k \leftarrow \text{DDIM}(z^k, \hat{\epsilon}^k, t)$
  $z^A \leftarrow \text{DDIM}(z^A, \hat{\epsilon}^A, t)$; free the step-$t$ cache

keep a self-injection exactly null in fp16 we compute $\text{SDPA}(Q,K,V) + \alpha\, W_{out}\big((P_{src} - P)V\big)$, which equals $\big((1-\alpha)P + \alpha P_{src}\big)V$ in exact arithmetic, with P_{src} stored in fp32.

Edit directions and the metric D. Clips A and B differ in subject and verb. In the *motion* direction the target prompt is B's subject with A's verb and we patch $\tilde{x}$, so the output should look like B and move like A. In the *content* direction the target prompt is A's subject with B's verb and we patch $\bar{x}$. We score content fidelity s_c as the mean per-frame CLIP image cosine [12] and motion fidelity s_m as the cosine of dense Farnebäck optical-flow fields [3] averaged over frame pairs. For the motion direction

$$D = \tfrac{1}{2}\big[\hat{s}_c(\text{out}, B) + \hat{s}_m(\text{out}, A)\big] - \tfrac{1}{2}\big[\hat{s}_c(\text{out}, A) + \hat{s}_m(\text{out}, B)\big], \tag{4}$$

with A and B exchanged for the content direction, and $\hat{s}$ min-max normalised over all values in a run. D rewards taking the right property from each clip and penalises taking the wrong one. The quantity we report is $\Delta D = D(\text{arm}) - D(\text{prompt only})$ on the same pair: what the patch adds over simply changing the prompt. The pre-registered success criterion (D-16) has three parts: in both directions, the median ΔD is positive, its 95% paired-bootstrap CI over pairs excludes zero, and subject consistency stays within 10% of the source clip's.

Motion LoRAs (the trained model). LoRA [7] adds a trainable low-rank update $\Delta W = \frac{\alpha}{r} BA$ to frozen weight matrices. We put LoRA on the query, key, value and output projections of the temporal attention layers, in three placements with the same rank, data and number of steps:

- `full`: all 21 motion modules (9.67 M trainable parameters);
- `probe`: only `up_blocks.0`, the H1 bootstrap set (1.97 M);
- `anti`: `down_blocks.2` and `.3`, the two blocks with the lowest $M(b)$ (2.62 M, more than `probe`), a wrong-place control.

Without the control, "`probe` is as good as `full`" could mean that any block would do. The loss is the standard ϵ-prediction objective of the frozen model,

$$\mathcal{L} = \mathbb{E}_{z_0,\epsilon,t}\left\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t,\ c)\right\|_2^2, \tag{5}$$

on 16-frame VAE latents z_0 of real clips with caption $c =$ "a person {verb}"; only the LoRA matrices receive gradients.

Interface. The brief asks for an interface where a user can test the method on their own inputs. We built a Gradio app (`scripts/app.py`) over an engine class (`src/vdprobe/demo.py`) that loads AnimateDiff, CLIP and the LoRAs once on the first request. It has four tabs.

- **Live edit.** The user picks clip A (subject, verb), clip B, a scene and a seed, a direction and a method: the selected component patch, a custom patch (any block, window and α), whole-tensor patch, attention injection, or prompt + LoRA. The engine runs A, B, the prompt-only arm and the edit in one lockstep group, and shows the four clips with s_c, s_m and D. A single edit has no pool of its own to normalise against, so D uses the pool of the scored test run, and the app says so. Subjects and verbs are free text.
- **Upload a video.** The user's video is cut to its longest shot (shot boundaries found from frame-difference jumps), resampled to 16 frames, encoded, and DDIM-inverted onto the sampler's 25 timesteps. The inverted trajectory then serves as the motion source for a new subject given in a prompt. The default method and starting noise for this path were chosen by experiment (Section 7.5).
- **Results browser.** Every clip of the dev and test sweeps with its scores and the run summary.
- **Where motion lives.** The S_{dyn} and S_{stat} curves for each coordinate and the H1–H3 verdicts.

Every live edit is saved with its GIFs, scores and the full request, so a result shown in the app can be reproduced.

5 Data set Details

Generated probing set (main data). The probing data are videos generated by the model itself, from the prompt template "a {subject} {verb} {scene}, high quality, detailed". Vocabulary, frozen on 22 Sep: 15 subjects in two body-plan families (quadrupeds: golden retriever, German shepherd, horse, wolf, tiger, fox, deer, brown bear; humans: young man, young woman, athlete, soldier, dancer, child, elderly man), 4 verbs (running, walking, jumping, spinning), 2 scenes (on a beach, in a forest), 3 seeds (42, 1337, 2024). That is 120 clips per seed and 360 in total, each 16 frames of 512×512 RGB. Within a (seed, scene) group every clip shares the same initial noise tensor, so two clips that differ in one word differ only because of that word. Pairs per seed: 104 content pairs (subject swapped within a family, verb fixed), 90 motion pairs (verb swapped, subject fixed) and 56 cross-family pairs, reported separately as a harder subset. Across the three seeds this gives 750 pairs and 300,000 distance rows (16 coordinates × 25 steps per pair). There is no download and no preprocessing beyond the model's own VAE decoding; the data are regenerated exactly from the configuration files.

Edit pairs. 12 test pairs and 4 separate dev pairs (`configs/edit_pairs.yaml`), frozen before any edit ran. In each pair A and B differ in both subject and verb; the test pairs have 6 per family, use every verb on both sides, and cycle through scenes and seeds. The dev pairs are used only to choose the patch configuration, so the test pairs never influence it.

HMDB51 clips for the LoRAs. Real human-action videos from HMDB51 [9], fetched from the Hugging Face mirror `divm/hmdb51` [24] at a pinned revision. HMDB51 has about 6,800 clips in 51 action classes, collected from films and web video; the clips we use are 240 pixels high at 30 fps. We use four classes that match our verbs (run, walk, jump, turn → spinning) and filter on HMDB's own metadata tags: full body visible, no camera motion, one person, quality not bad, at most one clip per source film. That leaves 19 clips (run 5, walk 5, jump 4, turn 5), 21 to 168 frames each, 2.0 MB in total, recorded with SHA-256 hashes in `data/hmdb51_lora/manifest.json`. The last clip of each class (sorted by name) is held out for validation: 15 train, 4 validation.

Preprocessing: 16 frames at stride $\min(2, \lfloor (n-1)/15 \rfloor)$ from a random start, a random square crop of full height, resizing to 256×256 with antialiasing, scaling to $[-1, 1]$, and encoding with the frozen VAE. Validation uses a fixed window and a centre crop. While debugging the upload path we found that 11 of the

19 clips contain a hard scene cut (a mean grey-level jump larger than both 4× the clip's median and 20). In the first recipe some training windows spanned a cut. The *cut-free* recipe (D-34) draws every window inside a single shot of at least 16 frames, chosen with probability proportional to its length; a clip without such a shot uses its longest shot with the last frame repeated.

6 Experiments

Hardware and software. A laptop RTX 4060 (8 GB, 15 GB RAM) for development, controls, LoRA training (first recipe), scoring and the interface; Kaggle sessions with 2× Tesla T4 (16 GB each) for the sweeps and the cut-free LoRAs, one job per GPU. Python 3.10, PyTorch 2.10 (2.11 on Kaggle), diffusers 0.40.0 [23], transformers 5.20, PEFT 0.21.2, scikit-learn 1.7.2, OpenCV, Gradio 6.30. Every result file records the software versions, GPU, timings, peak memory, and the resolved model repository and commit hash. One clip costs 65 s on the laptop and 90–97 s on a T4.

Code. `https://github.com/rujulaaaa/IE643`. The README lists the command for every stage. The suite of 51 CPU tests runs on a tiny random U-Net in about 10 s and covers hooks, patches, controls, the analysis on a synthetic sweep with a planted answer, the metrics and the interface engine.

E4. Lockstep verification. Four checks against `pipe()`: two `pipe()` runs (floor), a solo lockstep clip, a clip inside a three-clip group, and a clip with all probes attached. All four final latents matched exactly. A self-pair gave distances below 3×10^{-7}; a verb swap gave a median d of 0.555 on $\tilde{x}$ and 0.083 on $\bar{x}$.

E5. Probing sweep. Six lockstep groups of 60 clips (3 seeds × 2 scenes), 16 coarse coordinates × 25 steps. The coordinates are one representative per block: the last motion module of each of the 9 blocks, and the cross-attention of the last spatial transformer of each of the 7 blocks that have one (the last module is closest to the block output, and cross-attention is where the prompt enters the spatial path). Kaggle, 2×T4, 5.2 hours. Analysis on the CPU: 1,000 bootstrap resamples, probes in 836 s.

E6. Patching controls on the real model. Null patches (source = target) for every patch type; a misindexed null patch (the source's activation from two steps earlier), which *must* fail; an unpatched arm compared with the same clip generated as another group's source; and a symmetry check, A→B against B→A on `up_blocks.0` over the first third of the steps. The first run of these controls failed two of them. The component null patch gave a mean absolute difference of 10^{-2} (the fp16 identity issue above). Attention recording perturbed the source clip by 6×10^{-3}, because the recorder returned its own explicit-softmax output instead of the stock one. A second run left the null attention injection at 2.3×10^{-3}, which led to the SDPA + $\alpha W_{out}(\cdot)$ form. The third run is the run of record (Table 2).

E7. Dev sweep (configuration selection). 4 dev pairs, 51 arms each: component patches on the motion-module outputs of each of the 9 temporal blocks over two windows (first third of the steps, i.e. steps 0–8; all 25), on all 21 motion modules over four windows (one step, five steps, first third, all), both directions, plus whole-tensor and attention-injection references. Rule (D-31, fixed before scoring): per direction, the component arm with the largest median ΔD among those whose median subject-consistency ratio is ≥ 0.9, ties going to fewer modules. Kaggle, 2.6 h. Selected: motion, $\tilde{x}$ patch on `up_blocks.3` motion modules, steps 0–8 (dev median ΔD +0.080); content, $\bar{x}$ patch on `down_blocks.3` motion modules, steps 0–8 (+0.015).

E8. Test sweep. 12 test pairs, 18 clips each: source, B, and per direction the prompt-only arm, the selected patch, the selected patch at $\alpha \in \{0.25, 0.5, 0.75\}$, the whole-tensor patch at the same place; temporal attention injection over steps 0–8 in all 42 temporal attention layers; prompt + each of the three LoRAs. Kaggle, 10,017 s. The first submission failed before generating anything because the Kaggle image's `torchao` was incompatible with the newest PEFT; we pinned PEFT 0.21.2 and removed `torchao`. Scoring (VAE decode, CLIP, optical flow) ran on the laptop in 15 minutes, with 10,000 paired bootstrap resamples

per CI.

E9. Motion LoRAs, first recipe. Rank 32, $\alpha = 32$, Gaussian initialisation, AdamW (learning rate 5×10^{-5}, weight decay 10^{-2}), batch 1, 800 steps, timesteps uniform over the 1,000 training steps, fp16 frozen weights with fp32 LoRA weights under autocast, seed 0. The first attempt ran out of memory on the 8 GB GPU at step 1: diffusers' gradient checkpointing covers only the ResNets of this model, not the motion modules where the LoRA sits. We wrapped every motion module in non-reentrant checkpointing (same gradients); peak memory fell to 5.0 GB. 17–22 minutes per variant. Validation: the 4 held-out clips at 8 fixed timesteps $\times$ 2 fixed noise tensors, identical for every variant and for the untrained model.

E10. Motion LoRAs, cut-free with three seeds (D-34). The same recipe with cut-free windows, seeds 0, 1 and 2, nine LoRAs, on Kaggle (22–30 minutes each on a T4). On the 12 test pairs we generated one prompt + LoRA arm per LoRA and attention injection over all 25 steps (the best motion arm in the dev sweep, which the selection rule did not allow us to pick), regenerated the source, B and the prompt-only arm, and scored everything in one pool with E8. Kaggle, 2×T4: about 1.9 h of training and then 1.7–2.0 h of edits per GPU.

E11. Upload path. Interface checks on real HMDB51 clips, comparing three starting points for the new clip (the inverted noise, a unit-variance blend of the inverted and a Gaussian noise, Gaussian noise alone) and three methods (selected $\tilde{x}$ patch, attention injection over the first third, attention injection over all steps).

7 Results

7.1 Where content and motion live

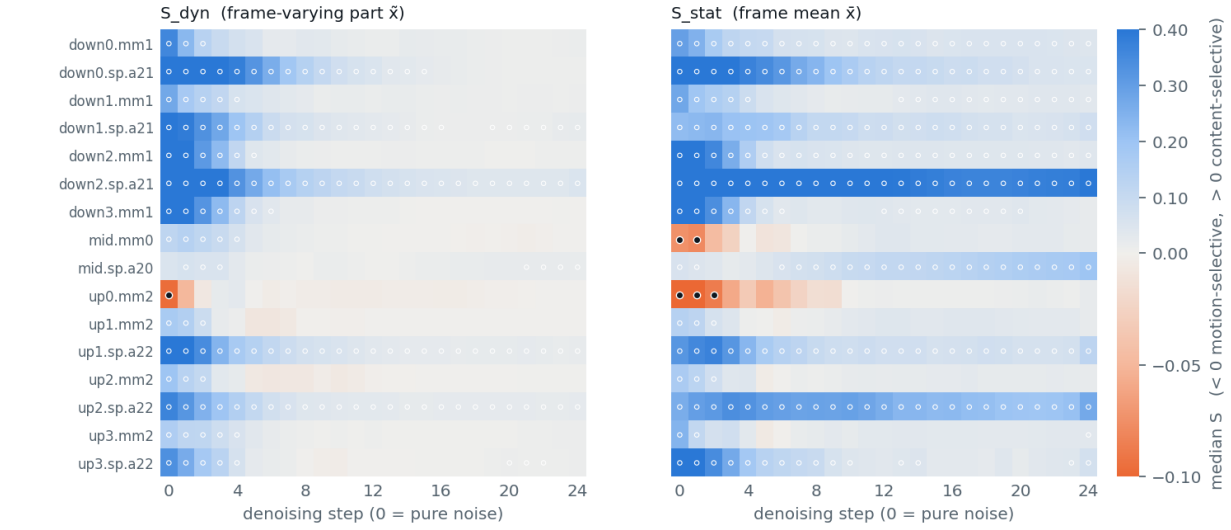


Figure 5: Selectivity maps over 16 coordinates and 25 denoising steps (`mm` = motion module, `sp.a2x` = spatial cross-attention). Left: S_{dyn} on the frame-varying part. Right: S_{stat} on the frame mean. Blue is content-selective, orange motion-selective. Only `up0.mm2` (and `mid.mm0` in $\bar{x}$) is motion-selective, and only in the first steps.

Figure 5 is the central result of the probing stage. Almost every cell is blue: a subject swap moves almost

every layer more than a verb swap does, including the temporal layers. The spatial cross-attention layers are the most content-selective. Every map fades towards zero after about step 9, where content and motion swaps move the activations by similar amounts. Table 1 gives the three pre-registered tests.

Hypothesis	Verdict	Evidence
H1: motion peaks at `up_blocks.1`	falsified	$M(b)$ is highest at `up_blocks.0` (+0.005), the argmax in 1000/1000 resamples. Every other block is negative: `up1` −0.017, `up2` −0.019, `mid` −0.026, `down0` −0.052, `down2` −0.075. The peak is sharp but at the extreme (8×8) resolution the test excluded.
H2: motion is settled early	survives	On `up0.mm2`, $m(t) = 0.096, 0.050, 0.007$, then about 0. Changepoint $\tau = 2$, 95% CI [1, 3]; early mean 0.073 vs. late −0.001.
H3: removing $\bar{x}$ reveals hidden motion	falsified	$f_{stat} = 0.65 < 0.75$ and no coordinate flips. $S_{dyn} < S_{stat}$ almost everywhere (e.g. `down2` cross-attention 0.43 → 0.17), so the split moves every coordinate towards motion, but never past zero.

Table 1: Pre-registered hypothesis tests (78 matched units per seed, 3 seeds, 1,000 bootstrap resamples).

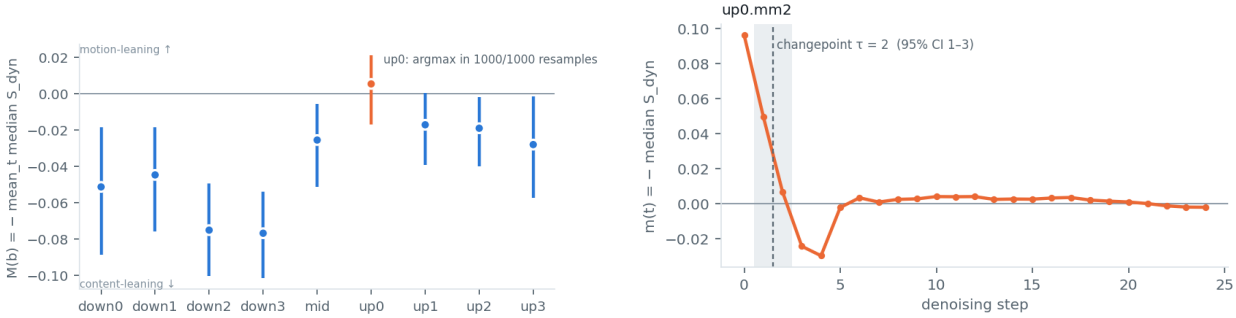


Figure 6: Left: motion selectivity $M(b)$ of the nine temporal blocks with 95% CIs; only `up_blocks.0` is above zero. Right: the motion-selectivity profile of `up0.mm2` over the denoising steps, with the fitted changepoint and its CI.

Two observations go beyond the tests. First, the H3 result is the opposite of what we expected: at the very first steps the cleaner motion signal is in $\bar{x}$, not $\tilde{x}$. S_{stat} has five motion-significant cells (`up0.mm2` at steps 0–2, `mid.mm0` at steps 0–1) and S_{dyn} only one. The static share of the activation energy, $F\|\bar{x}\|^2/\|x\|^2$, is 0.39–0.70 in the motion modules but 0.82–0.9996 in the spatial cross-attention layers, so "the frame mean is content" holds on the spatial path, not the temporal one. Second, on `up0.mm2` the motion profile dips below zero at steps 3–4 (−0.024, −0.030) before settling. Our two-segment test was not designed to detect that, and we have not interpreted it.

Robustness. Cosine and relative-L2 distances give nearly the same map (Spearman 0.987), and the median of per-unit ratios agrees with the ratio of medians (0.930). Seed-to-seed agreement of the S_{dyn} map is only moderate (0.58, 0.69, 0.73), so cell-level detail should not be over-read.

Linear probes. With the scene split, the verb is decodable well above chance (0.25) in places, for example 0.74 on the mid-block cross-attention (Figure 7). Much of this is the verb token leaking through the prompt: on that coordinate the probe on $\bar{x}$ stays at 1.0 at every step, because cross-attention reads the text embedding directly. On the temporal modules the frame mean also decodes the verb better than the residual at every step. Selectivity and decodability are different questions: a layer can carry the verb and still respond more to a change of subject.

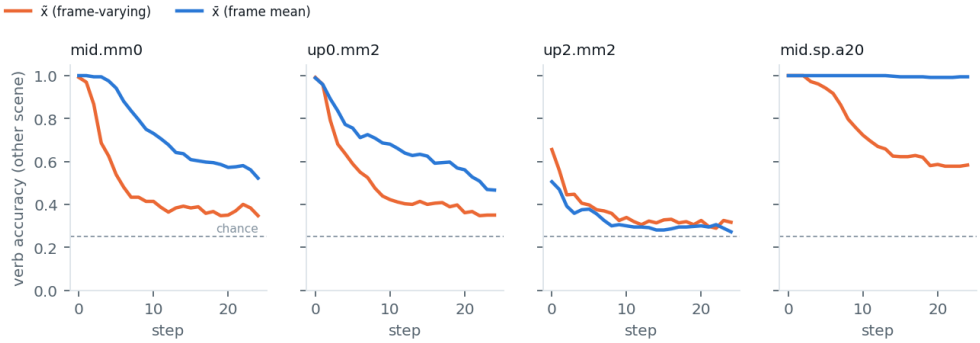


Figure 7: Verb-probe accuracy over the denoising steps on four coordinates, trained on one scene and tested on the other (chance 0.25). $\bar{x}$ decodes the verb at least as well as $\tilde{x}$ everywhere.

7.2 Patching controls

Control	Result	Verdict
Null patch: whole tensor, $\tilde{x}$ only, $\bar{x}$ only, attention injection	MAD 0, PSNR ∞	pass
Mis-indexed null patch (source from 2 steps earlier)	MAD 0.103, PSNR 25.1 dB	detected, as required
Unpatched arm vs. the same clip as another group's source	MAD 0	pass (no leakage)
Symmetry, `up_blocks.0`, steps 0–8: $f(A \to B)$ vs. $f(B \to A)$	0.171 vs. 0.083	fail (ratio 2.06 $>$ 2)

Table 2: Controls on the real model (run of record). MAD = mean absolute difference of the final latents.

The patching machinery is exact: every null patch reproduces the unpatched clip bit for bit, and a deliberately wrong patch is caught. The symmetry control fails our own criterion. The effect has the same sign both ways, but moving the golden retriever's run into the walking horse transfers twice as much as the reverse. One explanation is that a running clip has more motion to move; we tested only one pair, so this stays a conjecture.

7.3 Edits on the held-out test pairs

Direction	Arm	median D	median ΔD	$\Delta D > 0$	95% CI
keep A's motion	prompt only	0.149	0	–	–
	$\tilde{x}$ patch, `up_blocks.3`, steps 0–8 (selected)	0.195	+0.041	8/12	[−0.027, 0.143]
	$\tilde{x}$ patch at $\alpha = 0.5$	0.236	+0.038	9/12	includes 0
	whole-tensor patch, same place	0.074	−0.036	5/12	[−0.109, 0.047]
	attention injection, steps 0–8 (baseline)	0.269	**+0.105**	10/12	**[0.001, 0.204]**
keep A's content	prompt only	0.090	0	–	–
	$\bar{x}$ patch, `down_blocks.3`, steps 0–8 (selected)	0.198	+0.068	10/12	[−0.010, 0.158]
	$\bar{x}$ patch at $\alpha = 0.5$	0.199	+0.029	9/12	[0.001, 0.152]
	whole-tensor patch, same place	0.133	−0.026	4/12	[−0.080, 0.106]

Table 3: Test-sweep results (12 pairs). ΔD is the paired gain over the prompt-only arm; the CI is a 95% paired bootstrap of the difference in medians (10,000 resamples). Subject consistency stayed within 1.2% of the source for every patch arm.

Table 3 answers the editing question. The pre-registered criterion is **not met**: the selected component patches beat the prompt-only edit on the median in both directions, in 8/12 and 10/12 pairs, with consistency intact, but neither CI excludes zero. Three points are still clear.

- Component patches beat whole-tensor patches at the same site in both directions. The whole-tensor patch copies A wholesale: in the motion direction it reaches $s_m(\text{out}, A) = 0.82$, but its content also moves to A ($s_c(\text{out}, A) = 0.91$), so $\Delta D < 0$. The selected $\tilde{x}$ patch raises $s_m(\text{out}, A)$ from 0.23 to 0.67 while $s_c(\text{out}, B)$ drops only from 0.85 to 0.81. This is the entanglement the split was meant to remove, and it removes much of it.
- Where motion is *represented* and where patching *uses* it are different places. The map peaks at `up_blocks.0`; the best patch site in the dev sweep was the outermost, 64×64 decoder block `up_blocks.3`, where patching `up_blocks.0` gave only +0.030.
- Temporal attention injection, which needs no map at all, is the strongest motion edit and the only arm whose CI excludes zero, and only by 0.001.

Attention injection over all 25 steps (E10). In the dev sweep the best motion arm overall was attention injection over the whole trajectory (ΔD +0.151), which the selection rule excluded because it is not a component patch. We ran it on the test pairs in the LoRA run. That run regenerated the source, B and the prompt-only clip of every pair, and all three came out bit-identical to the test sweep's (mean absolute latent difference 0 on all 12 pairs), so the two runs can be scored in one pool. Over all steps, injection reaches median $\Delta D = +0.120$, positive on 11 of 12 pairs, with CI [0.020, 0.203]: the clearest positive result of the project. It raises $s_m(\text{out}, A)$ to 0.76 while $s_c(\text{out}, B)$ stays at 0.81. Adding the new arms changes the min-max pool, so the other arms move slightly (the selected $\tilde{x}$ patch +0.045, CI [−0.019, 0.148]; injection over steps 0–8 +0.108, CI [0.006, 0.202]); Table 3 keeps the numbers of record from the test run alone.

Figures 8 and 9 show one pair in both directions. They also show the limits of our metrics. CLIP similarity is a coarse measure of identity, and the flow cosine rewards matching *where* pixels move, which includes splashes and camera-like drift as well as the animal's legs.

7.4 Motion LoRAs

Variant	Params	Held-out loss at step 800 first recipe	Held-out loss at step 800 cut-free, seeds 0/1/2	Test pairs, prompt + LoRA, median ΔD first recipe	Test pairs, prompt + LoRA, median ΔD cut-free, per seed
untrained	–	0.0996	0.0982	–	–
`full`	9.67 M	0.0638	0.0627 / 0.0627 / 0.0627	−0.021	+0.004 / +0.008 / −0.013
`probe`	1.97 M	0.0652	0.0641 / 0.0641 / 0.0641	+0.023	+0.014 / +0.015 / +0.009
`anti`	2.62 M	0.0646	0.0635 / 0.0636 / 0.0635	−0.020	−0.011 / −0.031 / −0.001

Table 4: Motion LoRAs. The untrained loss differs between recipes because the cut-free validation window is the centre of the longest shot. Cut-free ΔD is scored in one pool with the test run (E10). Every ΔD CI in the table includes zero.

On held-out loss, placement matters less than size. In both recipes `full` is lowest and the wrong-place control `anti` ends *below* the map-guided `probe`, with 33% more parameters. The ordering is identical for all three cut-free seeds, and the spread between seeds is below 10^{-4}, so it is not noise. The validation loss is not specific to motion, though (Figure 10). The edit test is the better measure, and there the first-recipe `probe` LoRA had the only positive median ΔD of the three. The cut-free retrain repeats this: `probe` is



Figure 8: Motion direction on test pair 1 ("a golden retriever running" → "a horse walking", beach, seed 42). Target prompt: "a horse running". The prompt-only horse runs, but in its own way; with the $\tilde{x}$ patch it picks up the dog's splash and stride, and its flow similarity to A rises from 0.38 to 0.60. D falls slightly on this pair because some of A's appearance comes along (the pale mane and spray).

positive in all three seeds (+0.009 to +0.015, 6–7 of 12 pairs), while `full` and `anti` scatter around zero. So on the edit test the map-guided placement does come out ahead, consistently but by a small margin, and no single CI excludes zero. The raw scores qualify the gain. With `probe`, $s_m(\text{out}, A)$ *falls* from 0.23 to 0.14–0.17 while $s_m(\text{out}, B)$ falls further, from 0.17 to 0.06–0.08. The LoRA makes the clip move less like B rather than more like A. That is what we should expect: it learned generic HMDB51 running, walking, jumping and turning, not clip A's particular motion, so we compare the three LoRAs with one another and not with the patches.

What the full LoRA learns, block by block. For the `full` LoRA we computed the relative update $\|\Delta W\|_F/\|W\|_F$ per temporal block and ranked the nine blocks by it, by selectivity $M(b)$, by verb-probe accuracy on $\tilde{x}$ and by usefulness for patching (dev ΔD). The LoRA's largest updates follow the probe ranking (Spearman +0.83, $p = 0.005$), peaking at `up_blocks.0` where both the map and the probe peak; they do not follow patching usefulness (−0.27). The three low-resolution blocks have larger relative up-



Figure 9: Content direction on the same pair. Target prompt: "a golden retriever walking". The $\bar{x}$ patch on `down_blocks.3` keeps the source dog's build, coat and raised tail more closely than the prompt alone (s_c to A 0.87 $\rightarrow$ 0.91) while it walks.

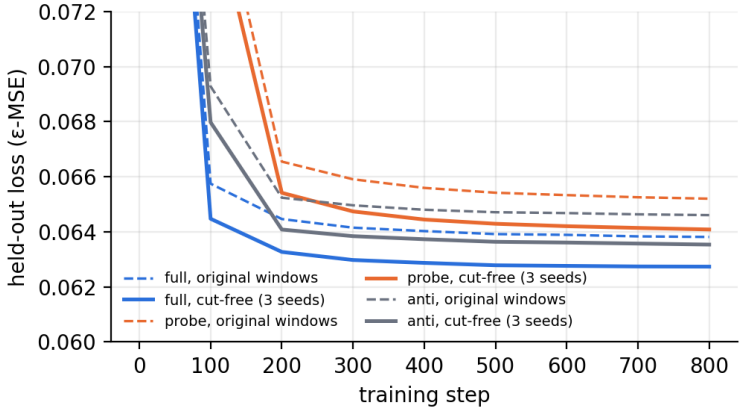


Figure 10: Held-out loss during training. Most of the drop happens in the first 100 steps for every variant, which looks more like adapting to HMDB51's look (grain, low resolution) than to its motion.

dates in general, so part of this agreement may be resolution rather than motion; we have not separated the two.

7.5 Upload path

New clip starts from	prompt only	selected $\tilde{x}$ patch	attention, steps 0–8	attention, all steps
inverted noise	0.118	0.519 (hybrid, artefacts)	0.309	**0.668**
blend of inverted and Gaussian	0.322	0.025 (diverges)	0.395	**0.656**
Gaussian noise	0.085	0.087 (diverges)	−0.053	0.366

Table 5: Uploaded video (a baseball pitch) turned into "a tiger jumping": flow cosine of the new clip to the upload. CLIP match to the target prompt is 0.33–0.36 for every clip that did not diverge.

The component patch does not survive this setting (Table 5). Logging norms inside the patcher showed why. For a generated pair the frame-varying parts of source and target already agree closely, since both start from the same noise, so the patch is a small nudge. For an uploaded video they are unrelated: the difference at `up_blocks.3` is up to 12,400 against an output norm of 8–21 thousand at step 0, the target's norm grows every step, and the final latent has a standard deviation of 26 instead of 0.7. The patch is a tool for pairs that share their initial noise, not a layout-free motion transplant. The interface therefore defaults to attention injection over all steps from the blended start. A second clip exposed another limit: a woman walking towards the camera became a scene that grows towards the camera, while the robot in it stayed small and nearly still. Temporal attention carries *where* things move in the frame, not *which object* moves.

7.6 Interface

All four tabs were checked through the running app with the same handlers as the buttons. A live edit (4 clips) takes 4.5–7 minutes on the laptop GPU at 6.4 GB peak; an upload edit about 4.5 minutes; the results browser and the maps need no GPU. Invalid combinations (attention injection in the content direction) are rejected with a message rather than an error.

8 Plan for Novelty Assessment

The results leave four open questions, and the plan follows them.

1. **Is the split an artefact of AnimateDiff's training recipe?** Its spatial layers were frozen while the motion module was trained, so content *had* to stay in the spatial path. We will run the same sweep (same vocabulary, same tests) on a model trained jointly on space and time, such as ModelScope T2V or VideoCrafter2. If H1–H3 come out the same, the map describes video diffusion; if not, it describes AnimateDiff.

2. **Combine what each method is good at.** Attention injection transfers *where* things move, and the $\tilde{x}$ patch carries *which* motion at the cost of some appearance. A combined arm, attention injection with a mask from the subject's cross-attention so that motion lands on the object, is the most direct fix for the failure in the upload path, and can be tested on the same 12 pairs.

3. **Statistical power.** Several arms have 9–10 of 12 positive pairs and CIs that miss zero by less than 0.03. We will add a second set of 24 pairs drawn by the same rule and report both sets, and run the symmetry control on all of them instead of one pair.

4. **Better measurements.** Replace optical-flow cosine with point tracks (CoTracker [8]) and CLIP with DINO features for identity, and separate the resolution effect in the LoRA update ranking by normalising per resolution level.

9 Conclusion

We asked where a video diffusion U-Net keeps content and where it keeps motion, using AnimateDiff and a controlled set of 750 one-word-swap pairs generated from shared noise. The answer is narrower than the architecture suggests. Content dominates almost every layer, temporal ones included. Motion is selective in only one place, the motion module just after the bottleneck, and only during the first two or three of 25 denoising steps. Removing the frame mean shifts every layer towards motion but reveals no hidden motion layer, and two of our three pre-registered hypotheses were falsified. Turning the same static/dynamic split into a patch gives edits that beat changing the prompt and beat whole-tensor patching in both directions, but not by a margin that 12 test pairs can confirm. Temporal attention injection did better, and over all 25 steps it is the one edit whose gain is clearly above zero. Where the map says motion is represented is not where patching works best. A LoRA placed by the map was not better on held-out loss than one placed where the map says motion is absent, but it was the only placement that improved the edit in all three seeds, by a small margin. The measurements are exact (null patches are bit-identical) and every decision was fixed before the data it affected, so the negative results are results. All code, the LoRAs and an interface for trying the edits are released with this report.

References

[1] Vatsal Baherwani, Yixuan Ren, and Abhinav Shrivastava. Characterizing motion encoding in video diffusion timesteps, arXiv preprint, 2025. https://arxiv.org/abs/2512.22175

[2] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023. https://arxiv.org/abs/2304.08818

[3] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion, Scandinavian Conference on Image Analysis (SCIA), 2003. https://doi.org/10.1007/3-540-45103-X_50

[4] Yuwei Guo, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin, and Bo Dai. AnimateDiff: Animate your personalized text-to-image diffusion models without specific tuning, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2307.04725

[5] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross attention control, International Conference on Learning Representations (ICLR), 2023. https://arxiv.org/abs/2208.01626

[6] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models, Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2204.03458

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models, International Conference on Learning Representations (ICLR), 2022. https://arxiv.org/abs/2106.09685

[8] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together, European Conference on Computer Vision (ECCV), 2024. https://arxiv.org/abs/2307.07635

[9] Hildegard Kuehne, Hueihan Jhuang, Estíbaliz Garrote, Tomaso Poggio, and Thomas Serre. HMDB: A large video database for human motion recognition, IEEE International Conference on Computer Vision (ICCV), 2011. https://doi.org/10.1109/ICCV.2011.6126543

[10] Pengyang Ling, Jiazi Bu, Pan Zhang, Xiaoyi Dong, Yuhang Zang, Tong Wu, Huaian Chen, Jiaqi Wang, and Yi Jin. MotionClone: Training-free motion cloning for controllable video generation, International Conference on Learning Representations (ICLR), 2025. https://arxiv.org/abs/2406.05338

[11] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT, Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2202.05262

[12] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, International Conference on Machine Learning (ICML), 2021. https://arxiv.org/abs/2103.00020

[13] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. https://arxiv.org/abs/2112.10752

[14] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, International Conference on Learning Representations (ICLR), 2021. https://arxiv.org/abs/2010.02502

[15] Raphael Tang, Linqing Liu, Akshat Pandey, Zhiying Jiang, Gefei Yang, Karun Kumar, Pontus Stenetorp, Jimmy Lin, and Ferhan Ture. What the DAAM: Interpreting Stable Diffusion using cross attention, Annual Meeting of the Association for Computational Linguistics (ACL), 2023. https://arxiv.org/abs/2210.04885

[16] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small, International Conference on Learning Representations (ICLR), 2023. https://arxiv.org/abs/2211.00593

[17] Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-A-Video: One-shot tuning of image diffusion models for text-to-video generation, IEEE/CVF International Conference on Computer Vision (ICCV), 2023. https://arxiv.org/abs/2212.11565

[18] Zeqi Xiao, Yifan Zhou, Shuai Yang, and Xingang Pan. Video diffusion models are training-free motion interpreter and controller, Advances in Neural Information Processing Systems (NeurIPS), 2024. https://arxiv.org/abs/2405.14864

[19] Danah Yatim, Rafail Fridman, Omer Bar-Tal, Yoni Kasten, and Tali Dekel. Space-time diffusion features for zero-shot text-driven motion transfer, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. https://arxiv.org/abs/2311.17009

[20] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2309.16042

[21] Rui Zhao, Yuchao Gu, Jay Zhangjie Wu, David Junhao Zhang, Jiawei Liu, Weijia Wu, Jussi Keppo, and Mike Zheng Shou. MotionDirector: Motion customization of text-to-video diffusion models, European Conference on Computer Vision (ECCV), 2024. https://arxiv.org/abs/2310.08465

[22] Yuwei Guo and others. https://huggingface.co/guoyww/animatediff-motion-adapter-v1-5-2, Accessed on: 20th October, 2026.

[23] Hugging Face. https://huggingface.co/docs/diffusers, Accessed on: 20th October, 2026.

[24] DIVM. https://huggingface.co/datasets/divm/hmdb51, Accessed on: 20th October, 2026.